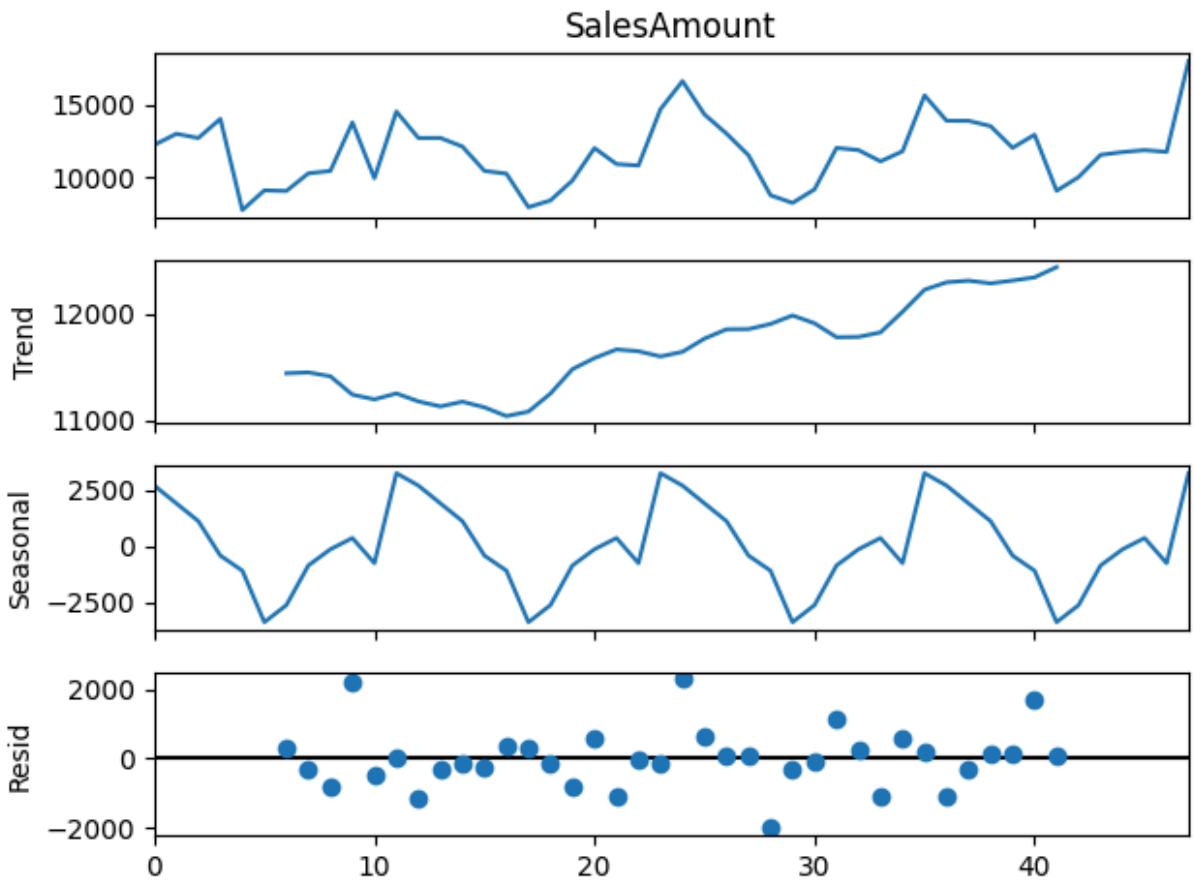
In [1]:
```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

In [13]:
```
np.random.seed(42)

start_date = datetime(2020, 1, 1)
num_months = 48  # 4 years of monthly data
date_rng = pd.date_range(start_date, periods=num_months, freq='MS')

sales_data = pd.DataFrame(date_rng, columns=['Date'])

# Base sales with a slight upward trend
base_sales = 10000 + np.arange(num_months) * 50

# Seasonal component (higher in Q4, lower in Q1)
seasonal_component = (
    2000 * np.sin(2 * np.pi * (sales_data['Date'].dt.month - 1) / 12 + np.pi/2)
    + 1500 * np.sin(2 * np.pi * (sales_data['Date'].dt.month - 1) / 6)
)

# Random noise
noise = np.random.normal(0, 500, num_months)

sales_data['SalesAmount'] = (base_sales + seasonal_component + noise).astype(int)
sales_data['SalesAmount'] = sales_data['SalesAmount'].clip(lower=2000)

# Add promotional flags (randomly)
sales_data['Promotion'] = np.random.choice([0, 1], num_months, p=[0.8, 0.2])
sales_data.loc[sales_data['Promotion'] == 1, 'SalesAmount'] = (
    sales_data.loc[sales_data['Promotion'] == 1, 'SalesAmount'] * np.random.uniform
).astype(int)

# Add holiday flags (e.g., December)
sales_data['HolidayMonth'] = (sales_data['Date'].dt.month == 12).astype(int)
sales_data.loc[sales_data['HolidayMonth'] == 1, 'SalesAmount'] = (
    sales_data.loc[sales_data['HolidayMonth'] == 1, 'SalesAmount'] * np.random.unif
).astype(int)

sales_data['SalesAmount'] = sales_data['SalesAmount'].astype(int)
```

In [14]:
```
decomposition = seasonal_decompose(sales_data['SalesAmount'], model='additive', per
decomposition.plot()
plt.tight_layout()
plt.show()
```



In [15]:
```
sales_data.to_csv('retail_sales_mock_data.csv', index=False)

print("Mock retail sales data generated: retail_sales_mock_data.csv")

sales_data.head()
```

```
Mock retail sales data generated: retail_sales_mock_data.csv
```

Out[15]:

	Date	SalesAmount	Promotion	HolidayMonth
0	2020-01-01	12248	0	0
1	2020-02-01	13011	0	0
2	2020-03-01	12722	0	0
3	2020-04-01	14030	1	0
4	2020-05-01	7783	0	0

Data Exploration

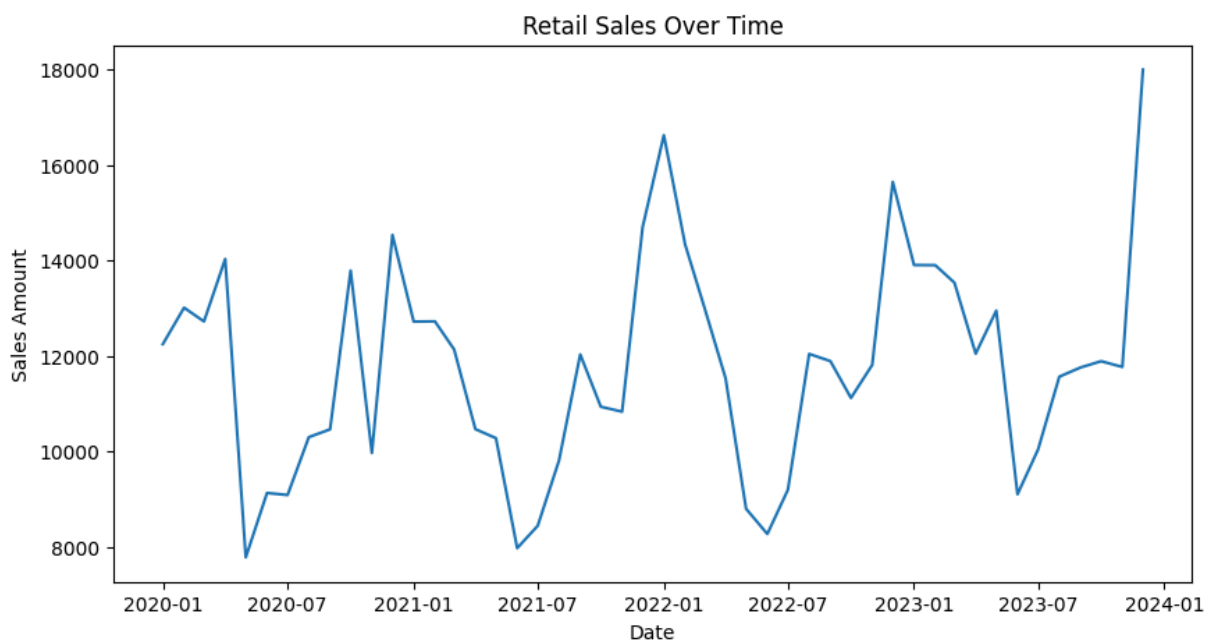
In [16]:
```
sales_data['Date'] = pd.to_datetime(sales_data['Date'])

sales_data.set_index('Date', inplace=True)

sales_data.head()
```

Out[16]:

Date	SalesAmount	Promotion	HolidayMonth
2020-01-01	12248	0	0
2020-02-01	13011	0	0
2020-03-01	12722	0	0
2020-04-01	14030	1	0
2020-05-01	7783	0	0

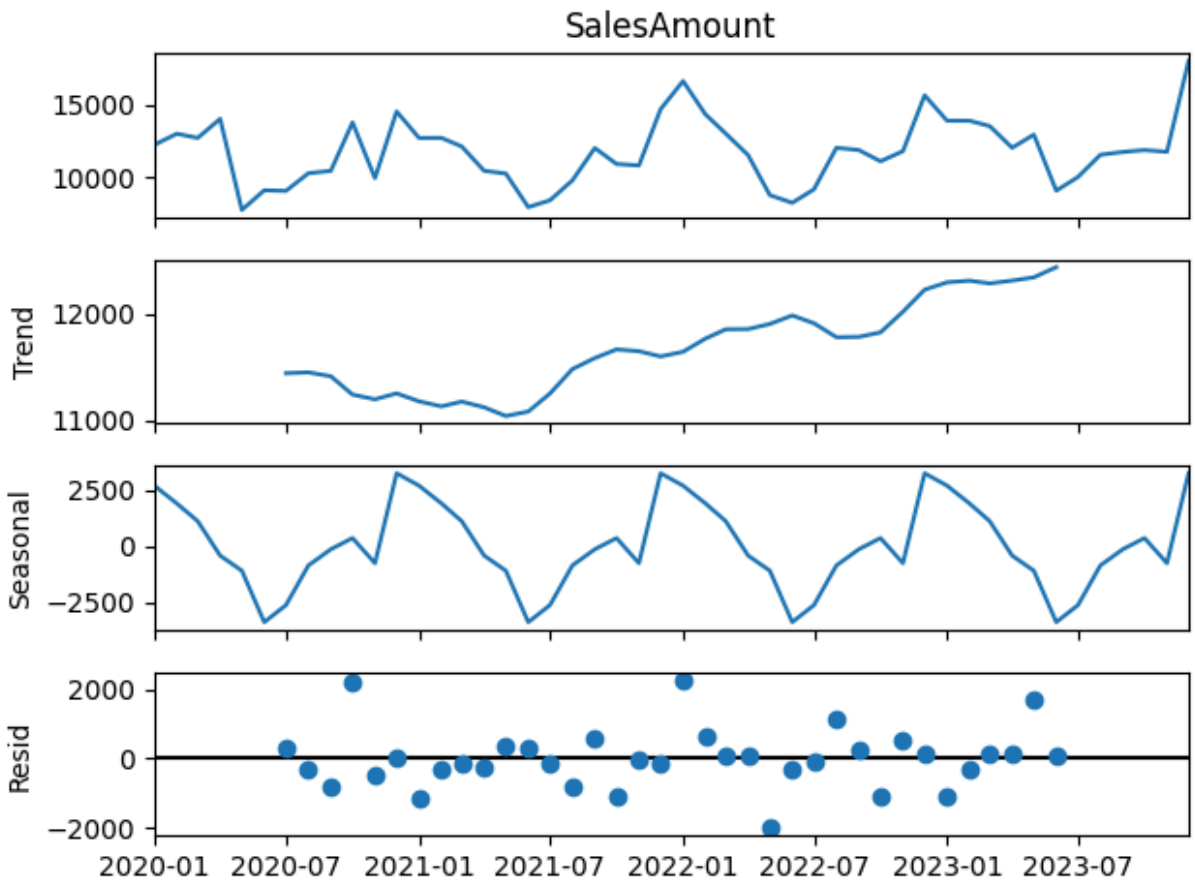
In [17]:

```
#visualization of sales over time
plt.figure(figsize=(10,5))
plt.plot(sales_data['SalesAmount'])
plt.title("Retail Sales Over Time")
plt.xlabel("Date")
plt.ylabel("Sales Amount")
plt.show()
```



In [18]:

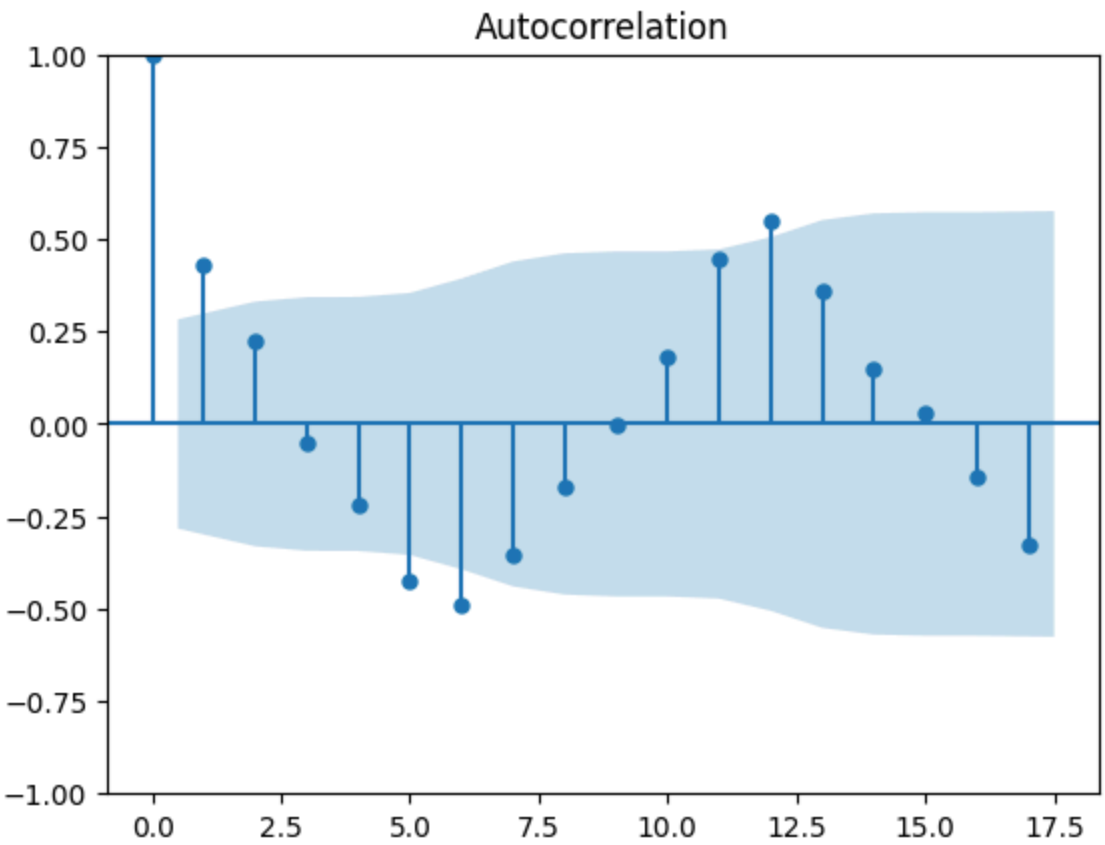
```
#Decomposition (Trend + Seasonality + Residuals)
decomposition = seasonal_decompose(sales_data['SalesAmount'], model='additive', per

decomposition.plot()
plt.show()
```

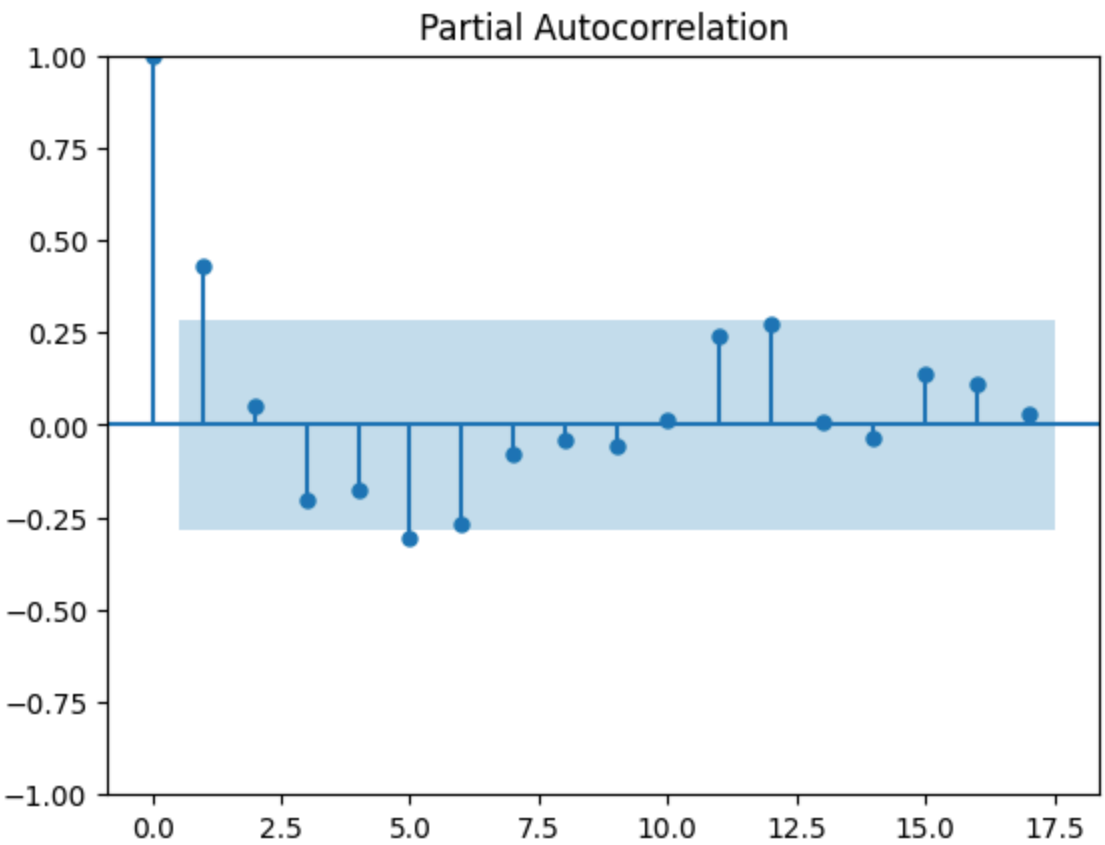


In [19]:
```
result = adfuller(sales_data['SalesAmount'])

print("ADF Statistic:", result[0])
print("p-value:", result[1])
```

```
ADF Statistic: -4.514183597733959
p-value: 0.0001853558643026136
```

In [20]:
```
plot_acf(sales_data['SalesAmount'])
plt.show()

plot_pacf(sales_data['SalesAmount'])
plt.show()
```





In [21]:
```
sales_data.isnull().sum()
```

Out[21]:
```
SalesAmount     0
Promotion       0
HolidayMonth    0
dtype: int64
```

In [22]:
```
# 80% training, 20% testing
train_size = int(len(sales_data) * 0.8)

train = sales_data[:train_size]['SalesAmount']
test = sales_data[train_size:]['SalesAmount']

print("Train size:", len(train))
print("Test size:", len(test))
```

```
Train size: 38
Test size: 10
```

In [23]:
```
# Fit SARIMA model
# (p,d,q) = (1,0,1) simple ARMA
# seasonal_order = (P,D,Q,s) with s=12 for monthly seasonality
model = SARIMAX(train,
                order=(1,0,1),
                seasonal_order=(1,1,1,12),
                enforce_stationarity=False,
                enforce_invertibility=False)

model_fit = model.fit()

print(model_fit.summary())
```

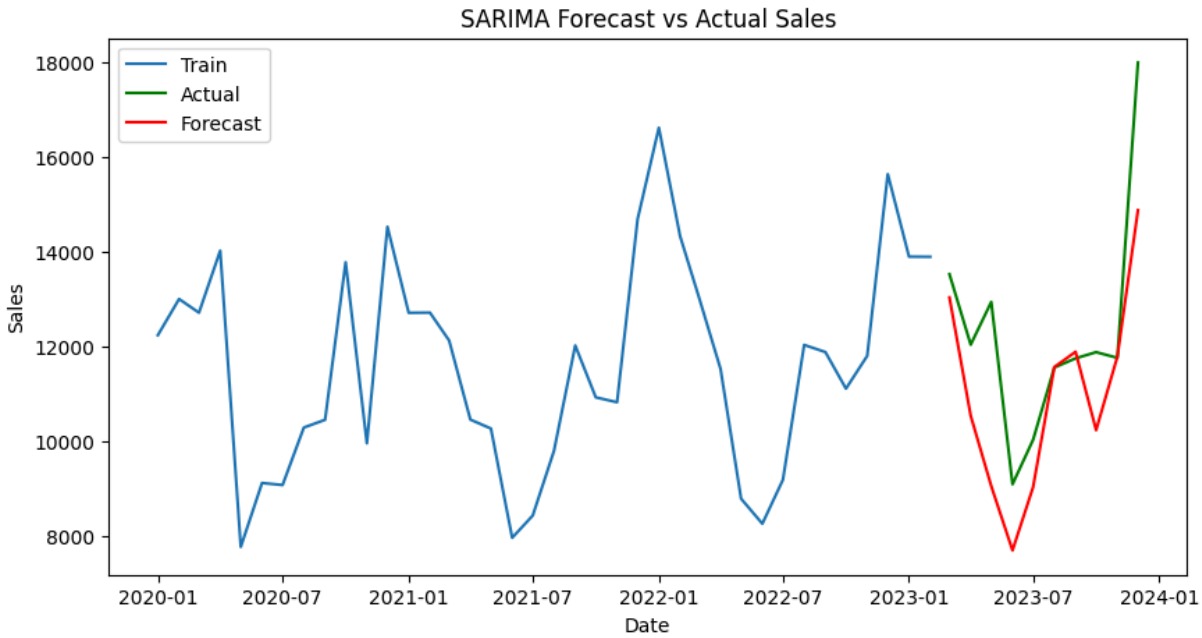
```
C:\Users\Hp\AppData\Roaming\Python\Python39\site-packages\statsmodels\tsa\base\tsa_m
odel.py:473: ValueWarning: No frequency information was provided, so inferred freque
ncy MS will be used.
  self._init_dates(dates, freq)
C:\Users\Hp\AppData\Roaming\Python\Python39\site-packages\statsmodels\tsa\base\tsa_m
odel.py:473: ValueWarning: No frequency information was provided, so inferred freque
ncy MS will be used.
  self._init_dates(dates, freq)
C:\Users\Hp\AppData\Roaming\Python\Python39\site-packages\statsmodels\tsa\statespace
\sarimax.py:866: UserWarning: Too few observations to estimate starting parameters f
or seasonal ARMA. All parameters except for variances will be set to zeros.
  warn('Too few observations to estimate starting parameters%s.'
```

```
                                     SARIMAX Results
==================================================================================
======
Dep. Variable:                        SalesAmount   No. Observations:
38
Model:             SARIMAX(1, 0, 1)x(1, 1, 1, 12)   Log Likelihood                -
97.449
Date:                            Fri, 27 Mar 2026   AIC                           2
04.898
Time:                                    07:08:32   BIC                           2
07.323
Sample:                                01-01-2020   HQIC                          2
04.000
                                     - 02-01-2023
Covariance Type:                              opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.0252      0.612     -0.041      0.967      -1.224       1.173
ma.L1          0.4271      0.462      0.924      0.355      -0.478       1.333
ar.S.L12      -0.6888      0.156     -4.412      0.000      -0.995      -0.383
ma.S.L12       1.2100      1.340      0.903      0.367      -1.416       3.837
sigma2      4.571e+05   8.31e+05      0.550      0.582   -1.17e+06    2.09e+06
===================================================================================
Ljung-Box (L1) (Q):                   0.45   Jarque-Bera (JB):                 1.29
Prob(Q):                              0.50   Prob(JB):                         0.52
Heteroskedasticity (H):               2.25   Skew:                             0.49
Prob(H) (two-sided):                  0.45   Kurtosis:                         1.73
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-ste
p).
```

```
C:\Users\Hp\AppData\Roaming\Python\Python39\site-packages\statsmodels\base\model.py:
607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check m
le_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
```

In [24]:

```python
# Forecast for test period
forecast = model_fit.forecast(steps=len(test))

plt.figure(figsize=(10,5))
plt.plot(train.index, train, label='Train')
plt.plot(test.index, test, label='Actual', color='green')
plt.plot(test.index, forecast, label='Forecast', color='red')
plt.title("SARIMA Forecast vs Actual Sales")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.legend()
plt.show()
```



In [25]:
```
mae = mean_absolute_error(test, forecast)
mse = mean_squared_error(test, forecast)
rmse = np.sqrt(mse)
mape = np.mean(np.abs((test - forecast) / test)) * 100

print("MAE:", round(mae,2))
print("RMSE:", round(rmse,2))
print("MAPE:", round(mape,2), "%")
```

```
MAE: 1319.75
RMSE: 1814.58
MAPE: 10.39 %
```

In [26]:
```
# Ensure the Date index is datetime and has monthly frequency
sales_data.index = pd.to_datetime(sales_data.index)
sales_data = sales_data.asfreq('MS')  # MS = Month Start

# Fit final SARIMA model on all historical data
final_model = SARIMAX(
    sales_data['SalesAmount'],
    order=(1,0,1),
    seasonal_order=(1,1,1,12),
    enforce_stationarity=False,
    enforce_invertibility=False
)

final_model_fit = final_model.fit(disp=False)
```

```
C:\Users\Hp\AppData\Roaming\Python\Python39\site-packages\statsmodels\tsa\statespace
\sarimax.py:866: UserWarning: Too few observations to estimate starting parameters f
or seasonal ARMA. All parameters except for variances will be set to zeros.
  warn('Too few observations to estimate starting parameters%s.'
```

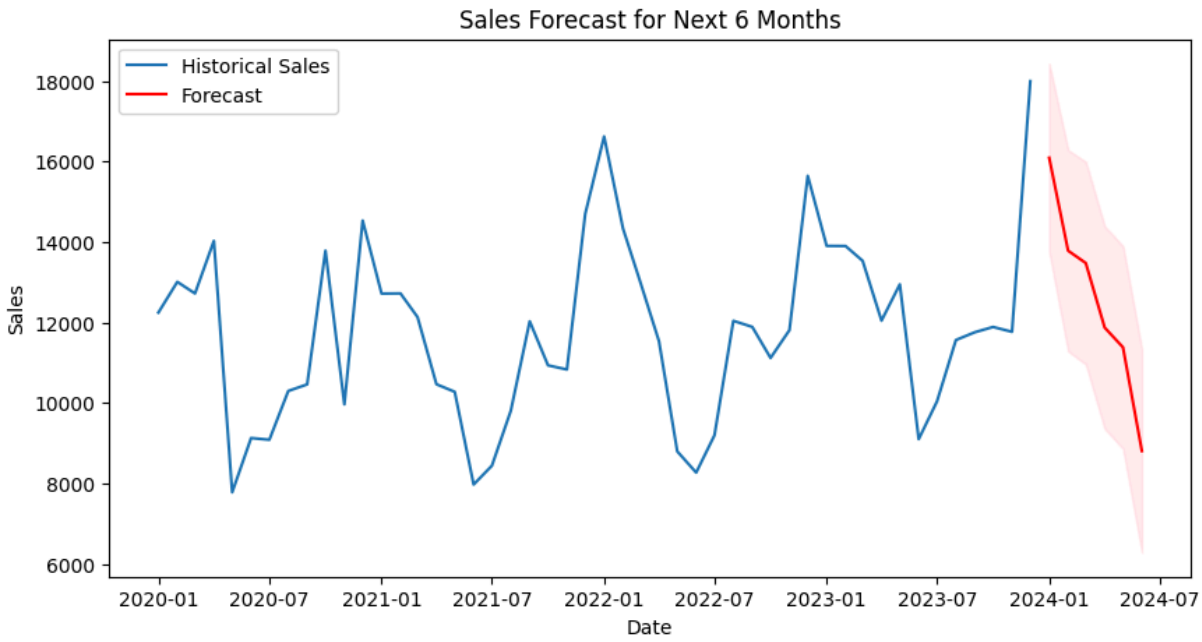
In [27]:
```
# Forecast 6 months ahead
future_forecast = final_model_fit.get_forecast(steps=6)
forecast_values = future_forecast.predicted_mean
```

```
confidence_intervals = future_forecast.conf_int()

print("Forecasted Sales:\n", forecast_values)
print("Confidence Intervals:\n", confidence_intervals)
```

```
Forecasted Sales:
 2024-01-01    16092.727436
2024-02-01    13785.140384
2024-03-01    13477.898981
2024-04-01    11874.550724
2024-05-01    11385.380793
2024-06-01     8810.105090
Freq: MS, Name: predicted_mean, dtype: float64
Confidence Intervals:
             lower SalesAmount  upper SalesAmount
2024-01-01       13755.188587       18430.266285
2024-02-01       11286.162731       16284.118037
2024-03-01       10964.313849       15991.484113
2024-04-01        9359.600093       14389.501355
2024-05-01        8870.302147       13900.459438
2024-06-01        6295.014440       11325.195740
```

In [28]:

```
plt.figure(figsize=(10,5))
plt.plot(sales_data['SalesAmount'], label="Historical Sales")
plt.plot(forecast_values.index, forecast_values, label="Forecast", color='red')

# Confidence interval shading
plt.fill_between(confidence_intervals.index,
                 confidence_intervals.iloc[:,0],
                 confidence_intervals.iloc[:,1],
                 color='pink', alpha=0.3)

plt.title("Sales Forecast for Next 6 Months")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.legend()
plt.show()
```


Sales Forecast for Next 6 Months
Historical Sales
Forecast
Sales
Date
18000
16000
14000
12000
10000
8000
6000
2020-01
2020-07
2021-01
2021-07
2022-01
2022-07
2023-01
2023-07
2024-01
2024-07

In [ ]:

In [ ]:

In [ ]:

In [ ]: